Lecture Notes: TwinCAT 3

Program Flow Instructions & State Machines

March 14, 2026

Contents

1 Introduction to Program Flow Control **2**

2 Conditional Statements **2**
- 2.1 IF / ELSIF / ELSE . . . 2
- 2.2 CASE Statements . . . 3

3 Iteration Statements (Loops) **4**
- 3.1 FOR Loops . . . 4
- 3.2 WHILE Loops . . . 6

4 State Machines and Best Practices **6**

5 Practical Implementation: CSV Persistent Storage **9**
- 5.1 Asynchronous Communication . . . 9
- 5.2 Helper Methods and String Manipulation . . . 10
- 5.3 Important Discovery . . . 12

Quick Reference Card **12**

1 Introduction to Program Flow Control

Instructions are used to control the flow of execution within a PLC program. There are four primary types of instructions covered:

- **Conditional Statements:** IF/ELSE and CASE switches.
- **Iteration Statements (Loops):** FOR loops and WHILE loops.

Category	Instruction	Use Case
Conditional	IF/ELSIF/ELSE	Boolean logic decisions
Conditional	CASE	Multiple discrete value testing
Iteration	FOR	Fixed number of iterations
Iteration	WHILE	Condition-based iteration

Table 1: Program Flow Instructions Overview

2 Conditional Statements

2.1 IF / ELSIF / ELSE

- **IF:** Specifies a block of code to execute only if a boolean expression evaluates to **TRUE**.
- **ELSE:** Provides an alternative path of execution if the IF condition is **FALSE**.
- **ELSIF:** Allows for testing additional conditions if the preceding conditions are false. You can use as many ELSIFs as needed.

Example: IF/ELSE Logic

```
// Program: Temperature Classification
// Demonstrates IF/ELSIF/ELSE conditional logic
PROGRAM MAIN
VAR
    nTemperature     : INT;            // Current temperature reading
    sStatus          : STRING(50);     // Status message
    bAlarmActive     : BOOL;           // Alarm flag

    // Threshold constants
    nHighThreshold   : INT := 80;      // High temperature threshold
    nLowThreshold    : INT := 20;      // Low temperature threshold
END_VAR

// IF/ELSIF/ELSE structure for temperature classification
IF nTemperature > nHighThreshold THEN
    // Temperature exceeds high threshold - critical condition
    sStatus := 'CRITICAL: Temperature too high!';
    bAlarmActive := TRUE;

ELSIF nTemperature < nLowThreshold THEN
    // Temperature below low threshold - warning condition
    sStatus := 'WARNING: Temperature too low!';
    bAlarmActive := TRUE;
```

```

ELSIF nTemperature = 50 THEN
    // Temperature exactly at optimal value
    sStatus := 'OPTIMAL: Perfect temperature';
    bAlarmActive := FALSE;

ELSE
    // Temperature within normal operating range
    sStatus := 'NORMAL: Temperature acceptable';
    bAlarmActive := FALSE;

END_IF
```

```
// Additional example: Nested IF statements
VAR
    bSensorOK        : BOOL;
    bSystemReady     : BOOL;
    nErrorCode       : INT;
END_VAR

IF bSensorOK THEN
    IF bSystemReady THEN
        // Both conditions met - proceed with operation
        nErrorCode := 0;
    ELSE
        // Sensor OK but system not ready
        nErrorCode := 2;
    END_IF
ELSE
    // Sensor failure - critical error
    nErrorCode := 1;
END_IF
```

2.2 CASE Statements

- **Purpose:** Used to perform different actions based on the specific value of a condition variable (typically an **INT** or **WORD**).
- **Note:** Strings are **not** supported in CASE statements; for strings, you must use IF/ELSIF.
- **Preferred Use:** Generally preferred over long chains of IF/ELSIF when testing many different values of a single variable.

Example: CASE Statement

```
// Program: State-Based Control using CASE
// Demonstrates CASE statement with integer state variable
PROGRAM MAIN
VAR
    nState           : INT := 1;       // Current state (1, 2, 3, etc.)
    sStateDesc       : STRING(50);     // State description
    bMotorEnable     : BOOL;           // Motor control output
    bValveOpen       : BOOL;           // Valve control output
    fSetpoint        : REAL;           // Process setpoint
END_VAR

```

```
// CASE statement for state machine control
CASE nState OF

    1:  // Initialization State
        sStateDesc := 'State 1: Initialization';
        bMotorEnable := FALSE;
        bValveOpen := FALSE;
        fSetpoint := 0.0;

        // Transition logic would go here
        // IF bInitComplete THEN nState := 2; END_IF

    2:  // Startup State
        sStateDesc := 'State 2: System Startup';
        bMotorEnable := TRUE;
        bValveOpen := FALSE;
        fSetpoint := 25.0;

    3:  // Running State
        sStateDesc := 'State 3: Normal Operation';
        bMotorEnable := TRUE;
        bValveOpen := TRUE;
        fSetpoint := 75.0;

    4, 5:  // Multiple values can share the same action
        sStateDesc := 'State 4 or 5: Maintenance Mode';
        bMotorEnable := FALSE;
        bValveOpen := FALSE;

    10..20:  // Range of values (10 through 20)
        sStateDesc := 'States 10-20: Error Recovery';
        bMotorEnable := FALSE;
        bValveOpen := FALSE;
        fSetpoint := 0.0;

ELSE
    // Default case - handles any undefined state values
    sStateDesc := 'UNKNOWN STATE - Check nState value';
    bMotorEnable := FALSE;
    bValveOpen := FALSE;
    fSetpoint := 0.0;

END_CASE
```

Warning: CASE statements do not support STRING variables. If you need to branch based on string values, you must use IF/ELSIF chains or convert the string to an enumeration or integer first.

3 Iteration Statements (Loops)

3.1 FOR Loops

- **Function:** Executes a block of code a specific number of times.
- **Use Case:** Highly efficient for iterating through **arrays**.

- **Mechanism:** Uses a counter variable that automatically increments by 1 (or a custom step size defined by the `BY` keyword) at the end of each loop until a target value is reached.

Example: FOR Loop Array Summation

```
// Program: Array Processing with FOR Loop
// Demonstrates summing array elements using a FOR loop
PROGRAM MAIN
VAR
    // Array of data values (indices 1 to 5)
    aData    : ARRAY[1..5] OF INT := [10, 20, 30, 40, 50];

    // Loop counter variable
    i        : INT;

    // Accumulator for sum
    nSum     : INT := 0;

    // Additional example variables
    nMax     : INT;
    nMin     : INT;
    fAverage: REAL;
END_VAR

// Reset sum before calculation
nSum := 0;

// FOR loop to sum all elements in the array
// Loop runs from index 1 to index 5 (inclusive)
FOR i := 1 TO 5 DO
    nSum := nSum + aData[i];
END_FOR

// Result: nSum = 10 + 20 + 30 + 40 + 50 = 150
```

```
// Additional FOR loop examples

// Example 1: Find maximum value in array
nMax := aData[1];  // Initialize with first element
FOR i := 2 TO 5 DO
    IF aData[i] > nMax THEN
        nMax := aData[i];
    END_IF
END_FOR

// Example 2: FOR loop with custom step size (BY keyword)
// Count down from 10 to 0 by steps of 2
VAR
    nCountdown : INT;
    j          : INT;
END_VAR

FOR j := 10 TO 0 BY -2 DO
    nCountdown := j;  // Values: 10, 8, 6, 4, 2, 0
END_FOR

// Example 3: Initialize array with calculated values
```

```
VAR
    aSquares : ARRAY[0..9] OF INT;
    k        : INT;
END_VAR

FOR k := 0 TO 9 DO
    aSquares[k] := k * k;  // Store square of index
END_FOR
// Result: aSquares = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

3.2 WHILE Loops

- **Function:** Continues to execute a block of code as long as a specified condition remains **TRUE**.
- **Risk:** Unlike FOR loops, the developer must ensure the condition eventually becomes false to avoid an infinite loop that could crash the PLC cycle.

```
// Example: WHILE loop for searching an array
VAR
    aBuffer     : ARRAY[0..99] OF INT;
    nSearchVal  : INT := 42;
    nFoundIndex : INT := -1;    // -1 indicates not found
    nIndex      : INT := 0;
    bFound      : BOOL := FALSE;
END_VAR

// Search for value using WHILE loop
nIndex := 0;
bFound := FALSE;

WHILE (nIndex <= 99) AND (NOT bFound) DO
    IF aBuffer[nIndex] = nSearchVal THEN
        nFoundIndex := nIndex;
        bFound := TRUE;
    ELSE
        nIndex := nIndex + 1;  // CRITICAL: Must increment to avoid infinite loop!
    END_IF
END_WHILE
```

> ⚠ **Warning:** WHILE loops can cause infinite loops if the exit condition is never met. Always ensure that the loop condition will eventually become FALSE, and consider adding a maximum iteration counter as a safety mechanism.

4 State Machines and Best Practices

In PLC programming, **state machines** are a standard way to manage complex sequences.

- **Avoiding "Magic Numbers":** Many programmers use integers (10, 20, 30) for states. The sources strongly advise **against** this because numbers do not describe what the code is doing.

- **Local Enumerations:** Use a **local ENUM** (defined within the function block) to name states (e.g., `FILE_OPEN`, `WAIT_FOR_EVENT`). This makes the code self-descriptive and prevents "namespace pollution".

Approach	Example	Readability
Magic Numbers	`CASE nState OF 10: ...`	Poor
Global ENUM	`CASE eState OF E_State.Running: ...`	Good
Local ENUM	`CASE eWriteState OF E_WriteState.FILE_OPEN: ...`	Best

Table 2: State Machine Implementation Approaches

Example: Local ENUM State Machine

```
// Function Block: FB_CsvEventWriter
// Demonstrates local ENUM for state machine implementation
FUNCTION_BLOCK FB_CsvEventWriter
VAR
    // LOCAL ENUMERATION - defined within the function block
    // Prevents namespace pollution and makes code self-descriptive
    {attribute 'strict'}
    E_WriteState : (
        IDLE,                   // Waiting for write request
        FILE_OPEN_TRIGGER,      // Trigger file open operation
        FILE_OPEN_WAIT,         // Wait for file to open
        WRITE_DATA_TRIGGER,     // Trigger data write
        WRITE_DATA_WAIT,        // Wait for write completion
        FILE_CLOSE_TRIGGER,     // Trigger file close
        FILE_CLOSE_WAIT,        // Wait for file to close
        WAIT_FOR_EVENT,         // Wait for next event to log
        DONE,                   // Operation completed successfully
        ERROR                   // Error state
    );

    // Current state variable using the local enum
    eWriteState : E_WriteState := E_WriteState.IDLE;

    // File operation function blocks
    fbFileOpen  : FB_FileOpen;
    fbFilePuts  : FB_FilePuts;
    fbFileClose : FB_FileClose;

    // File handle and path
    hFile       : UINT;
    sFilePath   : STRING(255) := 'C:\EventLogs\events.csv';

    // Status flags
    bBusy       : BOOL;
    bError      : BOOL;
    nErrorId    : UDINT;
END_VAR
```

```
// State machine implementation using CASE with local ENUM
CASE eWriteState OF

```

```
    E_WriteState.IDLE:
        // Initial state - waiting for trigger
        bBusy := FALSE;
        bError := FALSE;
        // Transition: IF bStartWrite THEN eWriteState :=
    E_WriteState.FILE_OPEN_TRIGGER; END_IF

    E_WriteState.FILE_OPEN_TRIGGER:
        // Trigger the file open operation
        bBusy := TRUE;
        fbFileOpen(
            bExecute := TRUE,
            sPathName := sFilePath,
            nMode := FOPEN_MODEAPPEND OR FOPEN_MODETEXT
        );
        eWriteState := E_WriteState.FILE_OPEN_WAIT;

    E_WriteState.FILE_OPEN_WAIT:
        // Wait for file open to complete
        fbFileOpen(bExecute := FALSE);

        IF NOT fbFileOpen.bBusy THEN
            IF fbFileOpen.bError THEN
                nErrorId := fbFileOpen.nErrId;
                eWriteState := E_WriteState.ERROR;
            ELSE
                hFile := fbFileOpen.hFile;
                eWriteState := E_WriteState.WRITE_DATA_TRIGGER;
            END_IF
        END_IF

    E_WriteState.WRITE_DATA_TRIGGER:
        // Trigger data write operation
        fbFilePuts(
            bExecute := TRUE,
            hFile := hFile,
            sLine := sCsvLine  // CSV formatted string
        );
        eWriteState := E_WriteState.WRITE_DATA_WAIT;

    E_WriteState.WRITE_DATA_WAIT:
        // Wait for write to complete
        fbFilePuts(bExecute := FALSE);

        IF NOT fbFilePuts.bBusy THEN
            IF fbFilePuts.bError THEN
                nErrorId := fbFilePuts.nErrId;
                eWriteState := E_WriteState.ERROR;
            ELSE
                eWriteState := E_WriteState.FILE_CLOSE_TRIGGER;
            END_IF
        END_IF

    E_WriteState.FILE_CLOSE_TRIGGER:
        // Trigger file close to flush data to disk
        fbFileClose(bExecute := TRUE, hFile := hFile);
        eWriteState := E_WriteState.FILE_CLOSE_WAIT;

```

```
    E_WriteState.FILE_CLOSE_WAIT:
        // Wait for file close to complete
        fbFileClose(bExecute := FALSE);

        IF NOT fbFileClose.bBusy THEN
            IF fbFileClose.bError THEN
                nErrorId := fbFileClose.nErrId;
                eWriteState := E_WriteState.ERROR;
            ELSE
                eWriteState := E_WriteState.DONE;
            END_IF
        END_IF

    E_WriteState.DONE:
        // Operation completed successfully
        bBusy := FALSE;
        eWriteState := E_WriteState.IDLE;

    E_WriteState.ERROR:
        // Error handling state
        bBusy := FALSE;
        bError := TRUE;
        // Remain in error state until acknowledged

END_CASE
```

Tip: Using the `{attribute 'strict'}` pragma on local enumerations ensures type safety and prevents accidental assignment of integer values directly to the enum variable.

5 Practical Implementation: CSV Persistent Storage

The lecture demonstrates these concepts by building a CSV event logger using a state machine and built-in Beckhoff function blocks (`FB_FileOpen`, `FB_FilePuts`, `FB_FileClose`).

5.1 Asynchronous Communication

- **Windows Dependency:** File operations are handled by Windows, not the real-time TwinCAT kernel. This communication is **asynchronous** and non-deterministic.
- **Trigger and Wait Pattern:** Because an operation might take several PLC cycles to complete, states must be split:
  1. **Trigger Step:** Set `bExecute := TRUE` to start the operation.
  2. **Wait Step:** Continuously call the FB and check the `bBusy` and `bError` flags until the operation is complete.

Step	Action	Next Step Condition
Trigger	Set `bExecute := TRUE`	Immediate
Wait	Check `bBusy` flag	`bBusy = FALSE`
Evaluate	Check `bError` flag	Proceed or Error state

Figure 1: Asynchronous Operation Pattern

5.2 Helper Methods and String Manipulation

- **CreateCsvString():** A private helper method used to convert a data structure into a comma-separated string.
- **TO_STRING Pragma:** Adding `{attribute 'to_string'}` to an enumeration allows TwinCAT to automatically convert the ENUM name into a string, which is useful for logging.
- **CONCAT:** A function used to join multiple strings together with commas and special characters like `$n` (new line).

Example: CSV String Building

```
// Enumeration with 'to_string' attribute for automatic string conversion
{attribute 'qualified_only'}
{attribute 'to_string'}
TYPE E_EventType :
(
    Info := 0,
    Warning := 1,
    Error := 2,
    Critical := 3
);
END_TYPE

{attribute 'qualified_only'}
{attribute 'to_string'}
TYPE E_EventSeverity :
(
    Low := 0,
    Medium := 1,
    High := 2
);
END_TYPE
```

```
// Private Method: CreateCsvString
// Converts an event structure into a comma-separated string for file storage
METHOD PRIVATE CreateCsvString : STRING(1000)
VAR_INPUT
    stEvent : REFERENCE TO ST_Event;  // Event to convert
END_VAR
VAR
    sTempString    : STRING(1000);     // Temporary string builder
    sEventType     : STRING(50);       // Event type as string
    sSeverity      : STRING(50);       // Severity as string
    sTimestamp     : STRING(50);       // Timestamp as string
    sEventId       : STRING(20);       // Event ID as string
END_VAR

// Convert enumeration values to strings using TO_STRING
// (Requires {attribute 'to_string'} on the enum definition)
sEventType := TO_STRING(stEvent.eEventType);
sSeverity := TO_STRING(stEvent.eEventSeverity);

```

```
// Convert numeric ID to string
sEventId := UDINT_TO_STRING(stEvent.nIdentity);

// Convert timestamp to string
sTimestamp := DT_TO_STRING(stEvent.dtTimestamp);

// Build CSV string using CONCAT
// Format: EventType,Severity,EventID,Timestamp,Description
sTempString := '';

// Add event type
sTempString := CONCAT(sTempString, sEventType);

// Add comma separator and severity
sTempString := CONCAT(sTempString, ',');
sTempString := CONCAT(sTempString, sSeverity);

// Add comma separator and event ID
sTempString := CONCAT(sTempString, ',');
sTempString := CONCAT(sTempString, sEventId);

// Add comma separator and timestamp
sTempString := CONCAT(sTempString, ',');
sTempString := CONCAT(sTempString, sTimestamp);

// Add comma separator and description text
sTempString := CONCAT(sTempString, ',');
sTempString := CONCAT(sTempString, stEvent.sText);

// Add newline character for end of CSV row
// $n is the escape sequence for newline in IEC 61131-3
sTempString := CONCAT(sTempString, '$n');

// Return the completed CSV string
CreateCsvString := sTempString;
```

```
// Example output from CreateCsvString:
// "Warning,High,1001,DT#2024-06-15-14:30:45,Pump pressure exceeded
    threshold$n"

// Alternative: More compact CONCAT chaining
sTempString := CONCAT(
    CONCAT(
        CONCAT(
            CONCAT(
                CONCAT(
                    CONCAT(sEventType, ','),
                    sSeverity
                ),
                ','
            ),
            sEventId
        ),
        ','
    ),
    CONCAT(
        CONCAT(sTimestamp, ','),
        CONCAT(stEvent.sText, '$n')
```

```
    )
);
```

5.3 Important Discovery

- **File Buffering:** In TwinCAT, data may not be written to the file immediately. The implementation found that explicitly calling `FB_FileClose` after writing an event was necessary to ensure the data was flushed to the disk.

> ⚠ **Warning:** File write operations in TwinCAT are buffered by Windows. To ensure data is actually written to disk (especially important for logging critical events), you must call `FB_FileClose` after writing. Simply calling `FB_FilePuts` does not guarantee the data reaches the physical file.

```
// Critical pattern: Always close file to flush buffer
// This ensures data is written to disk

// After write operation completes...
E_WriteState.FILE_CLOSE_TRIGGER:
    // IMPORTANT: Close file to flush write buffer to disk
    // Without this, data may remain in memory buffer and not be saved
    fbFileClose(bExecute := TRUE, hFile := hFile);
    eWriteState := E_WriteState.FILE_CLOSE_WAIT;
```

Quick Reference Card

Conditional Statement Syntax

```
// IF/ELSIF/ELSE
IF <condition1> THEN
    // Code block 1
ELSIF <condition2> THEN
    // Code block 2
ELSE
    // Default code block
END_IF

// CASE Statement
CASE <variable> OF
    <value1>:
        // Code for value1
    <value2>, <value3>:
        // Code for value2 or value3
    <start>..<end>:
        // Code for range
ELSE
    // Default code
END_CASE
```

Loop Syntax

```
// FOR Loop
FOR <counter> := <start> TO <end> BY <step> DO
    // Loop body
END_FOR

// WHILE Loop
WHILE <condition> DO
    // Loop body (must modify condition!)
END_WHILE
```

State Machine Best Practices

Practice	Recommendation
State Values	Use local ENUMs, not magic numbers
State Names	Descriptive (e.g., `FILE_OPEN_WAIT`)
Async Operations	Split into TRIGGER and WAIT states
Error Handling	Include dedicated ERROR state
File Operations	Always close files to flush buffers

Table 3: State Machine Best Practices

String Functions

Function	Purpose
`CONCAT(s1, s2)`	Join two strings
`TO_STRING(enum)`	Convert ENUM to string (requires attribute)
`INT_TO_STRING(n)`	Convert integer to string
`DT_TO_STRING(dt)`	Convert DATE_AND_TIME to string
`$n`	Newline escape sequence

Table 4: Common String Functions

Last Updated: March 14, 2026